Week 5 Lab Assignment.

Team: StudyHub
Members: Nazarbek(240311), Abdullox(240349), Usmon(240021)

## Architectural Justification for Project StudyHub

The architecture of StudyHub is strategically designed as a hybrid of Client-Server and Layered patterns to maximize system resilience, maintainability, and user experience. By implementing a Client-Server model at the container level (L2), the system establishes a clean boundary between the presentation layer and the core logic. This separation is critical for a university-scale application, as it allows the Go-based backend to serve as a centralized, secure gateway for sensitive operations, such as managing AWS S3 storage buckets and orchestrating calls to the Gemini LLM API. Utilizing a Reverse Proxy at the entry point further enhances this model by providing a unified interface for the student, managing SSL termination, and efficiently routing traffic between the static frontend and the dynamic API.

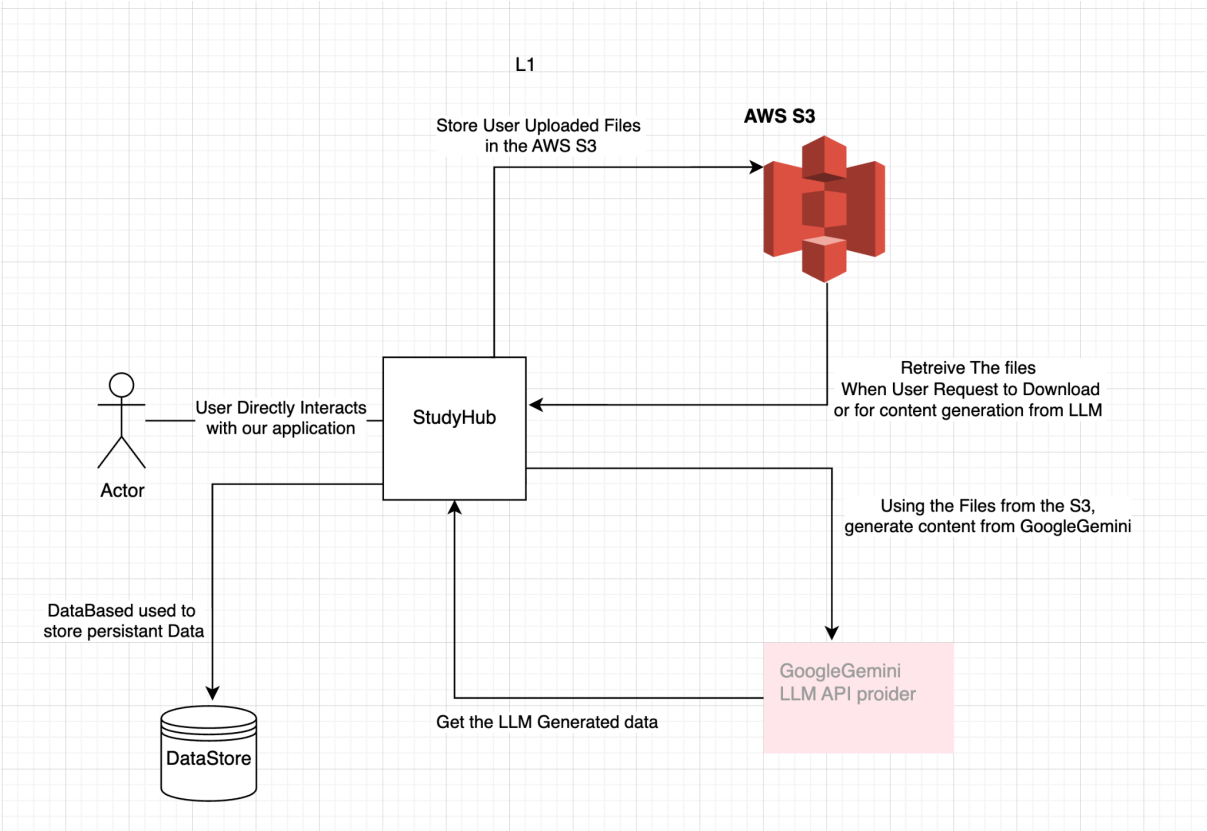
Internally, the backend follows a strict Layered Architecture (L3), which is essential for a project involving multiple evolving integrations. By decoupling the Transport Layer (HTTP/Routing), the Service Layer (Business Logic), and the Repository Layer (Data Access), the application achieves high modularity. This ensures that if the underlying database schema in PostgreSQL changes, or if the system migrates from Gemini to another LLM provider, the impact is isolated to a single layer rather than rippling through the entire codebase.

Furthermore, the architecture specifically addresses the performance challenges of generative AI through an Asynchronous Worker Pattern. By offloading intensive content generation tasks to a ContentGeneratorWorker via a Redis-backed queue, the system avoids the "blocking" cons typically associated with traditional layered and client-server models. This ensures that the student receives immediate feedback that their request is being processed, while the backend manages the latency of the external Gemini API in the background. This combination of patterns results in a robust, enterprise-ready structure that is both easy to debug and capable of scaling as more study modules are added to the platform.

## C4 diagrams.

**Context Level Diagram (Level 1):**

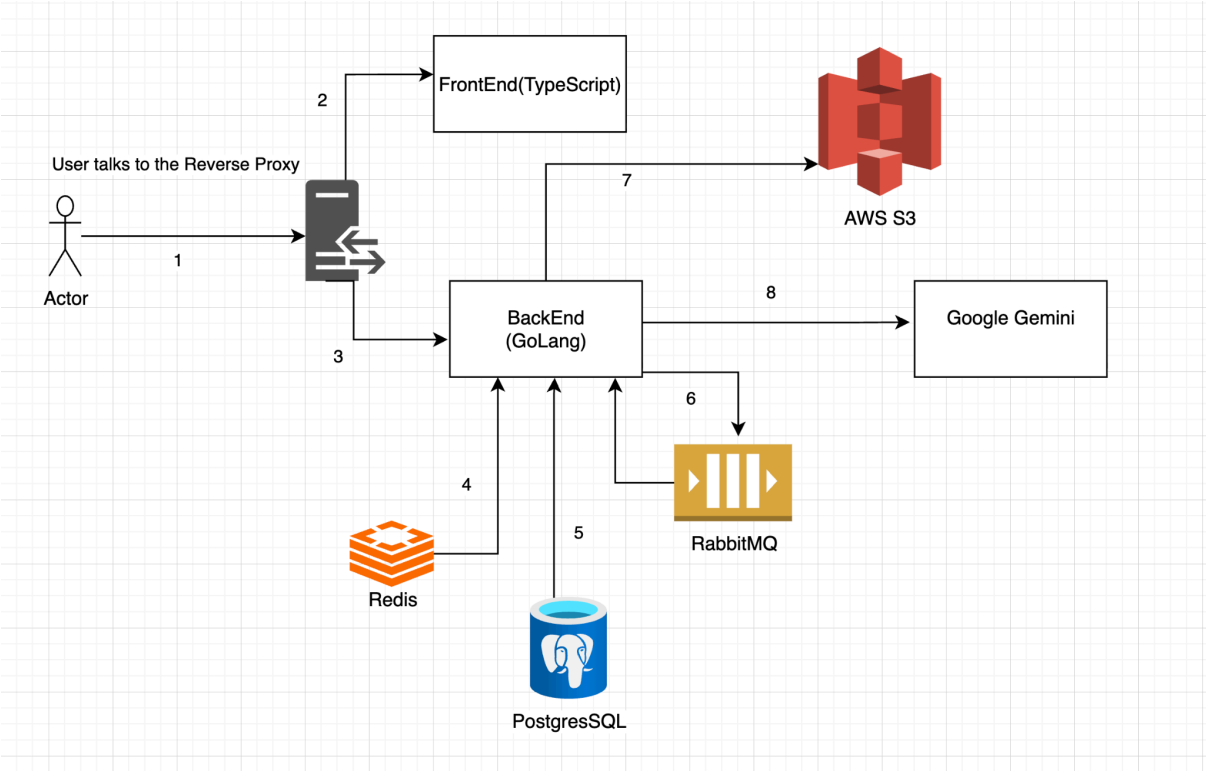
> **L1 System Context Diagram.** This diagram provides a black-box view of the StudyHub ecosystem. It identifies the functional scope of the system by mapping the flow of data between the user and three primary external entities: AWS S3 (for file management), the Google Gemini API (for AI processing), and the DataStore (for persistence). The diagram establishes that StudyHub acts as the central orchestrator for these services without exposing internal technical implementation details.




## L2 Container Request Lifecycle

1. **User Initiation:** The Actor (student) initiates a request by interacting with the platform. This traffic first hits the Reverse Proxy, which acts as the secure entry point for the entire system.
2. **Static Content Routing:** The Reverse Proxy identifies requests for the user interface and routes them to the FrontEnd (TypeScript) container. This serves the web application assets to the student's browser.
3. **API Routing:** Requests requiring dynamic data or processing (such as file uploads or AI generation) are routed by the Reverse Proxy to the BackEnd (Go) container.
4. **Session & Speed Layer:** The BackEnd interacts with Redis to manage transient data. This typically includes caching frequent queries or managing user session states to reduce latency.
5. **Primary Persistence:** Core relational data—such as user profiles, module metadata, and study records is stored in and retrieved from the PostgreSQL database.
6. **Asynchronous Task Queuing:** For intensive operations like AI content generation, the BackEnd publishes a message to RabbitMQ. This decouples the request from the processing, ensuring the user isn't stuck waiting for the API to respond.
7. **Object Storage Interaction:** The system communicates with AWS S3 to store raw study materials uploaded by the user or to retrieve files needed for context.

8. **AI Orchestration:** The BackEnd (or a worker listening to the queue) sends the necessary context to the Google Gemini LLM API to generate synthesized study insights and receives the processed data back.




## L3 Component Logic and Data Flow

The L3 Component Diagram (Figure 3) illustrates the internal decomposition of the BackEnd (Go) container into three distinct logical layers: **Transport**, **Service**, and **Repository**. This layered approach ensures a clean separation of concerns and facilitates modular development.

### 1. Transport Layer: The Entry Point

- **HttpLayer / Routing:** This component acts as the interface to the outside world. It receives incoming requests from the Reverse Proxy (defined in L2), parses the request payload, validates the input, and routes the task to the appropriate service in the next layer.

### 2. Service Layer: The Business Logic "Brain"

This layer contains the core application logic and coordinates high-level workflows.

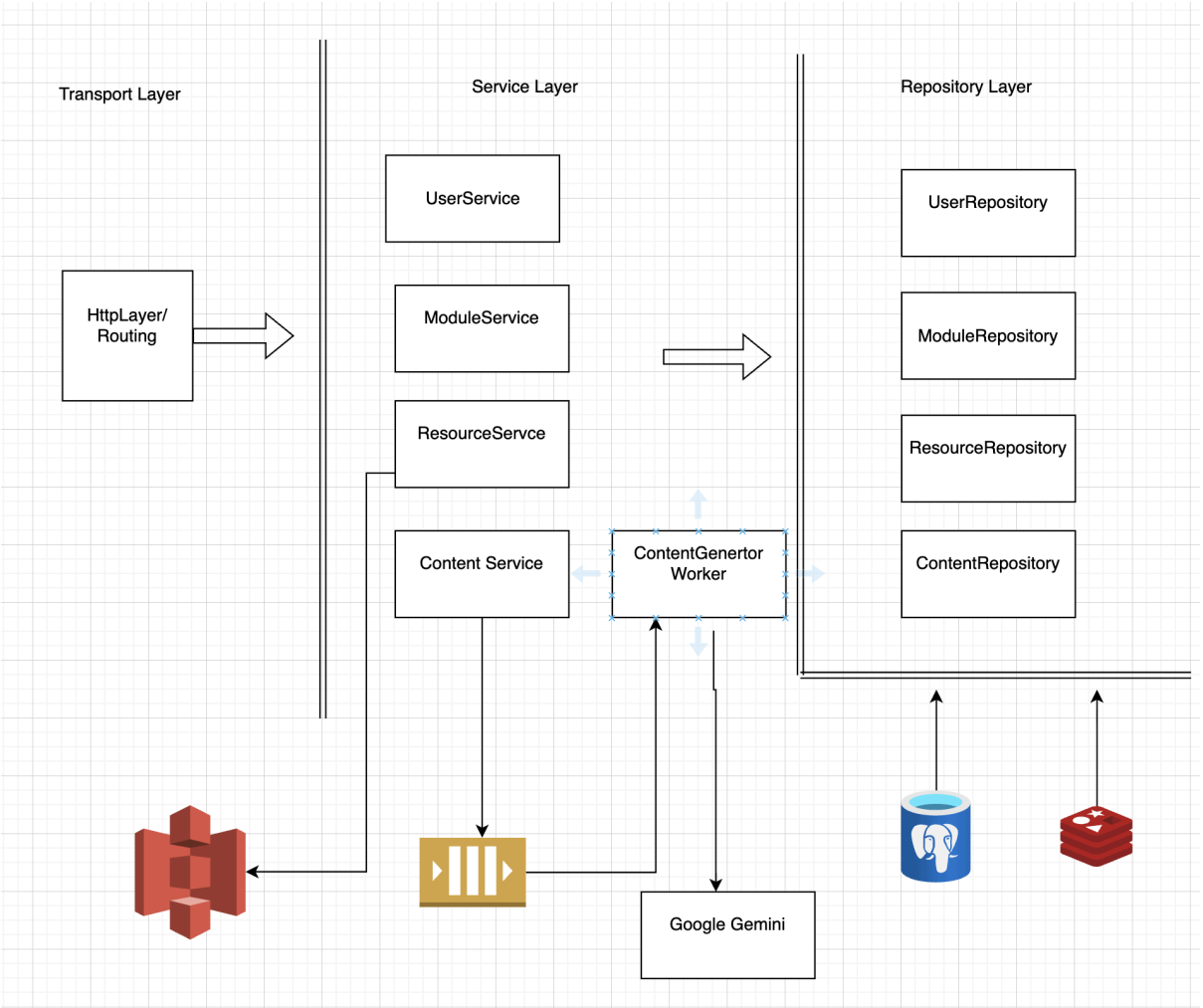
- UserService**:** Manages student profiles and authentication logic.
- ModuleService**:** Coordinates the organization of study modules and subjects.
- ResourceService**:** Directly manages file-related logic, interacting with AWS S3 to handle the storage and retrieval of study materials.

- **Content Service:** Acts as the orchestrator for AI generation. When a request for study content is made, it publishes a message to the RabbitMQ queue to initiate asynchronous processing.
- **ContentGeneratorWorker:** An autonomous component that consumes tasks from the queue. It fetches the necessary file context, communicates with the Google Gemini API, and processes the resulting AI data.

### 3. Repository Layer: Data Access & Persistence

This layer isolates the application's business logic from the underlying storage technology.

- **Individual Repositories (User, Module, Resource, Content):** These components are responsible for translating domain-level requests into specific database queries.
- **External Interaction:** The repository layer manages all outbound communication to the PostgreSQL DataStore for long-term persistence and the Redis Speed Layer for caching and session management.



# Technology Selection & Justification

## 1. Backend Language: Go (Golang)

- **Selected:** Go was chosen for the core backend logic.
- **Justification:** Go provides excellent support for concurrency, which is vital for managing multiple simultaneous user requests and background worker processes like the `ContentGeneratorWorker`. Its compiled nature offers high performance with a lower memory footprint compared to interpreted languages.
- **Alternatives Considered:** Python was considered for its AI libraries; however, Go was selected for its superior performance in distributed systems and its ability to handle asynchronous tasks more efficiently in a production environment.

## 2. Database: PostgreSQL

- **Selected:** PostgreSQL serves as the primary relational DataStore.
- **Justification:** It provides robust support for complex relational data, ensuring ACID compliance for student profiles and module metadata. It is highly extensible and integrates seamlessly with Go's repository layer.
- **Alternatives Considered:** NoSQL options like MongoDB were considered for document storage, but PostgreSQL was preferred because StudyHub relies on structured relationships between users, modules, and resources.

## 3. Caching & Speed Layer: Redis

- **Selected:** Redis is used for transient data and session management.
- **Justification:** Redis acts as a "speed layer" to reduce database load and cache expensive Gemini API responses, significantly improving system responsiveness.
- **Alternatives Considered:** Local in-memory caching was considered, but Redis was selected to allow the backend to scale horizontally across multiple containers while maintaining a shared state.

## 4. Message Broker: RabbitMQ

- **Selected:** RabbitMQ is implemented for asynchronous task queuing.
- **Justification:** It decouples the long-running AI generation process from the main request-response cycle. This prevents the backend from blocking while waiting for the Gemini API, ensuring the student's UI remains responsive.
- **Alternatives Considered:** Synchronous API calls were considered but rejected because the latency of LLM processing would lead to frequent timeouts and a poor user experience.

## 5. Object Storage: AWS S3

- **Selected:** AWS S3 for file persistence.

- **Justification:** S3 provides highly durable and scalable storage for student-uploaded study materials, removing the burden of file system management from the application server.
- **Alternatives Considered:** Local disk storage was considered for development, but AWS S3 was chosen for the final architecture to ensure cloud-native scalability and data durability.

## 6. Web Server & Reverse Proxy: Nginx

- **Selected:** Nginx was implemented as the entry point for all system traffic.
- **Justification:** Nginx acts as a high-performance **Reverse Proxy**, providing a unified interface for the user while handling SSL termination and load balancing. It efficiently routes traffic to the static React frontend (Step 2) or the Go API (Step 3), ensuring the backend is never directly exposed to the public internet.
- **Alternatives Considered:** Apache was considered, but Nginx was selected for its superior event-driven architecture, which handles high concurrency with significantly lower resource consumption.

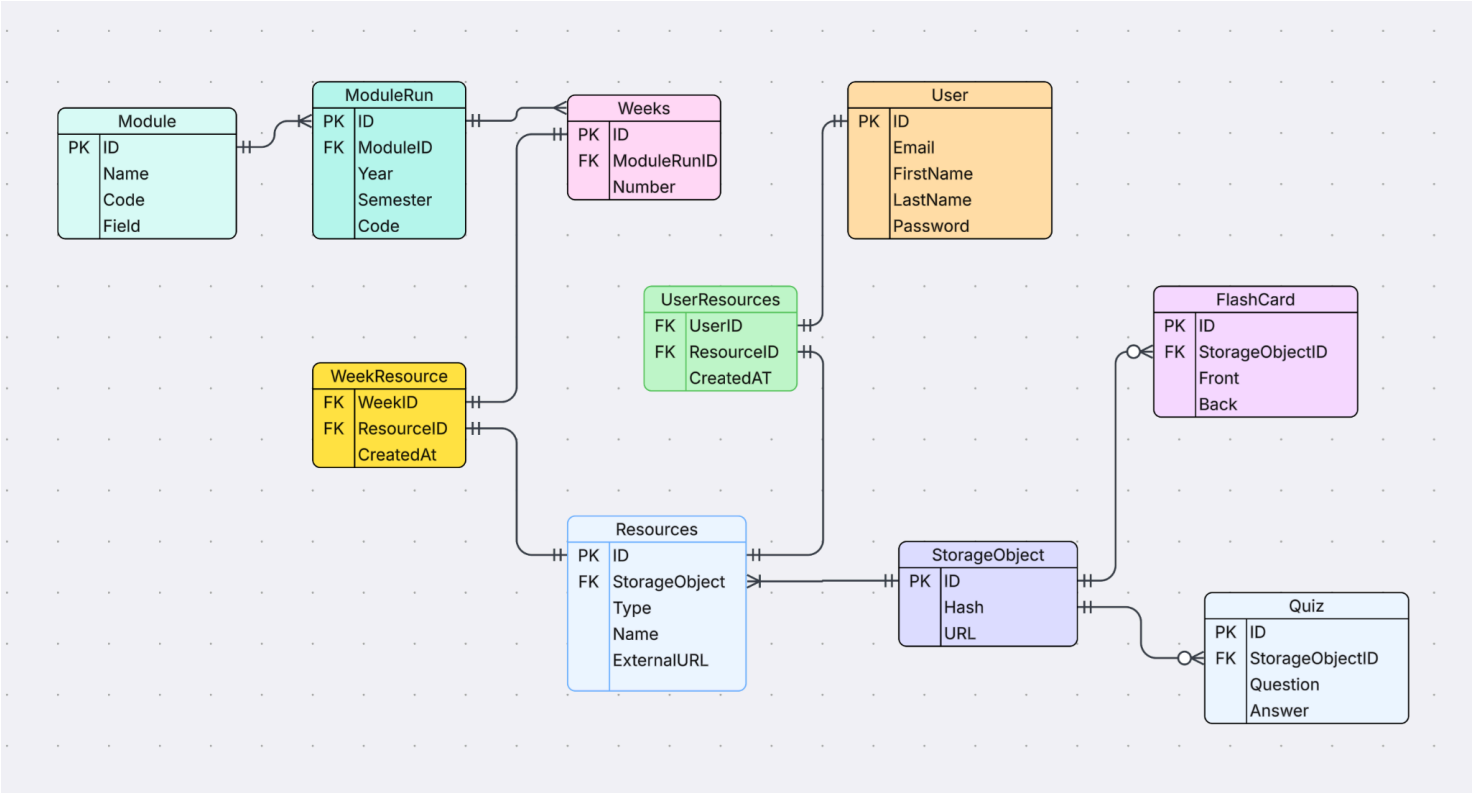
## 7. Frontend Framework: React (TypeScript)

- **Selected:** React was chosen for the client-side interface.
- **Justification:** React follows a Component-Based Architecture, which aligns with the modular nature of StudyHub's study modules. Using TypeScript adds static typing, reducing runtime errors and improving the maintainability of the complex state involved in file uploads and AI content rendering.
- **Alternatives Considered:** Vue.js was considered for its simplicity; however, React was selected due to its vast ecosystem of libraries (like Axios for API calls and Tailwind for styling) and its proven performance in large-scale web applications.

## Summary of Architectural Patterns

| Pattern | Selection | Justification |
|---|---|---|
| **Overall Model** | **Client-Server** | Provides clear separation of concerns and centralized control of resources like S3 and Gemini. |
| **Internal Structure** | **Layered** | Organizes code into Transport, Service, and Repository layers to maximize maintainability and testability. |

## DATA MODEL




# Entity-Relationship Data Model Description

The StudyHub data model is designed to manage educational resources, user interactions, and AI-generated study aids (Flashcards and Quizzes) through a relational structure. The model ensures data integrity and supports the complex mapping between academic modules and digital resources. In all tables the highest possible form of Normalization is achieved.

1. **Core Academic Entities**

- Module: Represents a high-level academic subject. It contains static metadata such as the Name, unique Code, and the academic Field.
- ModuleRun: Tracks specific instances of a Module (e.g., Spring 2026). It links to a ModuleID and includes time-bound data like Year, Semester, and a specific Code for that run.
- Weeks: Represents the chronological organization of a ModuleRun. Each week is associated with a ModuleRunID and identifies the specific Number in the academic calendar.

2. **User & Resource Management**

- User: Stores primary actor information, including Email, FirstName, LastName, and an encrypted Password.

- Resources: Acts as a logical pointer for study materials. It tracks metadata such as the file Type, Name, and an ExternalURL (referencing the AWS S3 location).
- StorageObject: Represents the physical manifestation of a resource. It includes a Hash for integrity checking and the direct URL for retrieval.

3. **AI-Generated Content (Output)**

- FlashCard: Contains AI-synthesized study prompts. Each card belongs to a StorageObjectID and consists of a Front (question) and Back (answer).
- Quiz: Represents assessment data generated from the resources. It links to a StorageObjectID and stores specific Question and Answer pairs.

---

**Entity Relationships**

The model utilizes several "Many-to-Many" junction tables to facilitate flexible resource sharing:

- Academic Hierarchy (One-to-Many): A single Module can have multiple ModuleRuns, which in turn contain multiple Weeks. This hierarchy allows resources to be pinned to specific points in a semester.
- Resource Mapping (Many-to-Many):
    - UserResources: Maps Users to Resources, allowing multiple students to access the same materials while tracking when they were first "CreatedAT" for that user.
    - WeekResource: Links Weeks to Resources, enabling a single document to be assigned to multiple weeks across different modules.
- Storage Attribution (One-to-Many): A StorageObject can serve as the foundation for multiple FlashCards or Quizzes, representing the AI-processed output derived from a single physical file.
- Resource-Storage Link: Every Resource record points to exactly one StorageObject, separating the logical metadata of a file from its physical storage properties.

# API Schemas

## User Routes

**Endpoint:** `GET /users`

- **Description:** Retrieve a paginated list of all users.
- **Authentication:** Bearer token required
- **Request Params:** `limit` (query, optional), `offset` (query, optional)
- **Response 200:** `[{ "id": "...", "email": "...", "firstName": "...", "lastName": "..." }]`
- **Response 401/403:** `{ "error": "Unauthorized" }`

**Endpoint:** `GET /users/me`

- **Description:** Retrieve the profile of the currently authenticated user.
- **Authentication:** Bearer token required
- **Response 200:** `{ "id": "...", "email": "...", "firstName": "...", "lastName": "..." }`
- **Response 401:** `{ "error": "Unauthorized" }`

**Endpoint:** `GET /users/{id}`

- **Description:** Retrieve a specific user profile by ID.
- **Authentication:** Bearer token required
- **Request Params:** `id` (path): User identifier
- **Response 200:** `{ "id": "...", "email": "...", "firstName": "...", "lastName": "..." }`
- **Response 404:** `{ "error": "User not found" }`

**Endpoint:** `DELETE /users/{id}`

- **Description:** Delete a user account.
- **Authentication:** Bearer token required (Admin only)
- **Request Params:** `id` (path): User identifier
- **Response 204:** No Content
- **Response 404:** `{ "error": "User not found" }`

---

## Module Routes

**Endpoint:** `GET /modules`

- **Description:** Retrieve a list of all high-level academic modules.
- **Authentication:** Bearer token required
- **Response 200:** `[{ "id": "...", "name": "...", "code": "...", "field": "..." }]`

**Endpoint:** `POST /modules`

- **Description:** Create a new academic module.
- **Authentication:** Bearer token required
- **Request Body:** `{ "name": "Computer Science", "code": "CS101", "field": "Engineering" }`
- **Response 201:** `{ "id": "...", "name": "...", "code": "...", "field": "..." }`
- **Response 400:** `{ "error": "Invalid input data" }`

**Endpoint:** `GET /modules/{id}`

- **Description:** Retrieve full details of a module, likely including its nested runs or weeks based on your `GetModuleFullHandler`.
- **Authentication:** Bearer token required
- **Request Params:** `id` (path): Module identifier
- **Response 200:** `{ "id": "...", "name": "...", "code": "...", "runs": [...] }`
- **Response 404:** `{ "error": "Module not found" }`

**Endpoint:** `DELETE /modules/{id}`

- **Description:** Delete a module and its associated runs.
- **Authentication:** Bearer token required
- **Request Params:** `id` (path): Module identifier
- **Response 204:** No Content

---

## Module Run Routes

**Endpoint:** `GET /modules/{moduleID}/runs`

- **Description:** List all specific semester runs for a given module.
- **Authentication:** Bearer token required
- **Request Params:** `moduleID` (path): Parent Module identifier
- **Response 200:** `[{ "id": "...", "moduleID": "...", "year": 2026, "semester": "Spring", "code": "..." }]`

**Endpoint:** `POST /modules/{moduleID}/runs`

- **Description:** Create a new run (instance) for a module.
- **Authentication:** Bearer token required
- **Request Params:** `moduleID` (path): Parent Module identifier
- **Request Body:** `{ "year": 2026, "semester": "Spring", "code": "CS101-SP26" }`
- **Response 201:** `{ "id": "...", "moduleID": "...", ... }`

**Endpoint:** `GET /module-runs/{id}`

- **Description:** Retrieve a specific module run by its direct ID.
- **Authentication:** Bearer token required
- **Request Params:** `id` (path): Module Run identifier
- **Response 200:** `{ "id": "...", "year": 2026, "semester": "Spring", "code": "..." }`

**Endpoint:** `DELETE /module-runs/{id}`

- **Description:** Delete a specific module run.
- **Authentication:** Bearer token required
- **Request Params:** `id` (path): Module Run identifier
- **Response 204:** No Content

---

## Academic Term Routes

**Endpoint:** `GET /academic-terms/current`

- **Description:** Retrieve the currently active academic term (simplifies querying for the active semester).
- **Authentication:** Bearer token required
- **Response 200:** `{ "id": "...", "year": 2026, "semester": "Spring", "isActive": true }`

**Endpoint:** `POST /academic-terms/new-term`

- **Description:** Create a new academic term and optionally set it as current.
- **Authentication:** Bearer token required
- **Request Body:** `{ "year": 2026, "semester": "Fall", "isActive": true }`
- **Response 201:** `{ "id": "...", "year": 2026, "semester": "Fall" }`

---

## Resource Routes

**Endpoint:** `POST /resources/file/{week_id}`

- **Description:** Upload a physical file to S3 and create a resource record linked to a specific week.
- **Authentication:** Bearer token required
- **Request Params:** `week_id` (path): The Week identifier
- **Content-Type:** `multipart/form-data`
- **Request Body:** `file` (binary), `name` (string)
- **Response 201:** `{ "id": "...", "type": "file", "name": "...", "externalURL": "s3://..." }`
- **Response 500:** `{ "error": "Failed to upload to S3" }`

**Endpoint:** `POST /resources/link/{week_id}`

- **Description:** Create a resource that points to an external link (like a YouTube video or article) for a specific week.
- **Authentication:** Bearer token required
- **Request Params:** `week_id` (path): The Week identifier

- **Request Body:** `{ "name": "...", "externalURL": "https://..." }`
- **Response 201:** `{ "id": "...", "type": "link", "name": "...", "externalURL": "..." }`

**Endpoint:** `GET /resources/{id}`

- **Description:** Retrieve details of a specific resource.
- **Authentication:** Bearer token required
- **Request Params:** `id` (path): Resource identifier
- **Response 200:** `{ "id": "...", "type": "...", "name": "...", "externalURL": "..." }`
- **Response 404:** `{ "error": "Resource not found" }`

**Endpoint:** `DELETE /resources/{id}`

- **Description:** Delete a resource (and trigger S3 deletion if it's a physical file).
- **Authentication:** Bearer token required
- **Request Params:** `id` (path): Resource identifier
- **Response 204:** No Content

**Endpoint:** `GET /resources/weeks/{week_id}`

- **Description:** List all resources assigned to a specific week.
- **Authentication:** Bearer token required
- **Request Params:** `week_id` (path): The Week identifier
- **Response 200:** `[{ "id": "...", "type": "...", "name": "...", "externalURL": "..." }]`

# Quality Attribute Scenarios

## Performance

- **Scenario:** A student uploads a 20-page PDF and requests a generated quiz during peak hours.
- **Response:** The Go Backend accepts the file, stores it in AWS S3, and offloads the processing task to RabbitMQ. The ContentGeneratorWorker processes the AI request asynchronously.
- **Measure:** The user receives a "Processing" confirmation in under 2 seconds, while the full AI-generated result is ready in the background within 30 seconds, preventing a browser timeout.

## Security

- **Scenario:** An unauthorized user attempts to access another student's uploaded study resources.

- **Response:** The Nginx Reverse Proxy forwards the request to the Go Backend, where the UserService and ResourceRepository perform an ownership check against the PostgreSQL database.
- **Measure:** The system returns a 403 Forbidden error, and the database ensures that only the user linked in the UserResources table can access the specific ResourceID.

### Availability

- **Scenario:** The primary PostgreSQL database undergoes a brief connection flicker or maintenance.
- **Response:** The BackEnd utilizes Redis to serve cached session data and previously generated content.
- **Measure:** The student can still view their existing dashboard and previously generated flashcards while the system re-establishes the connection to the main DataStore.

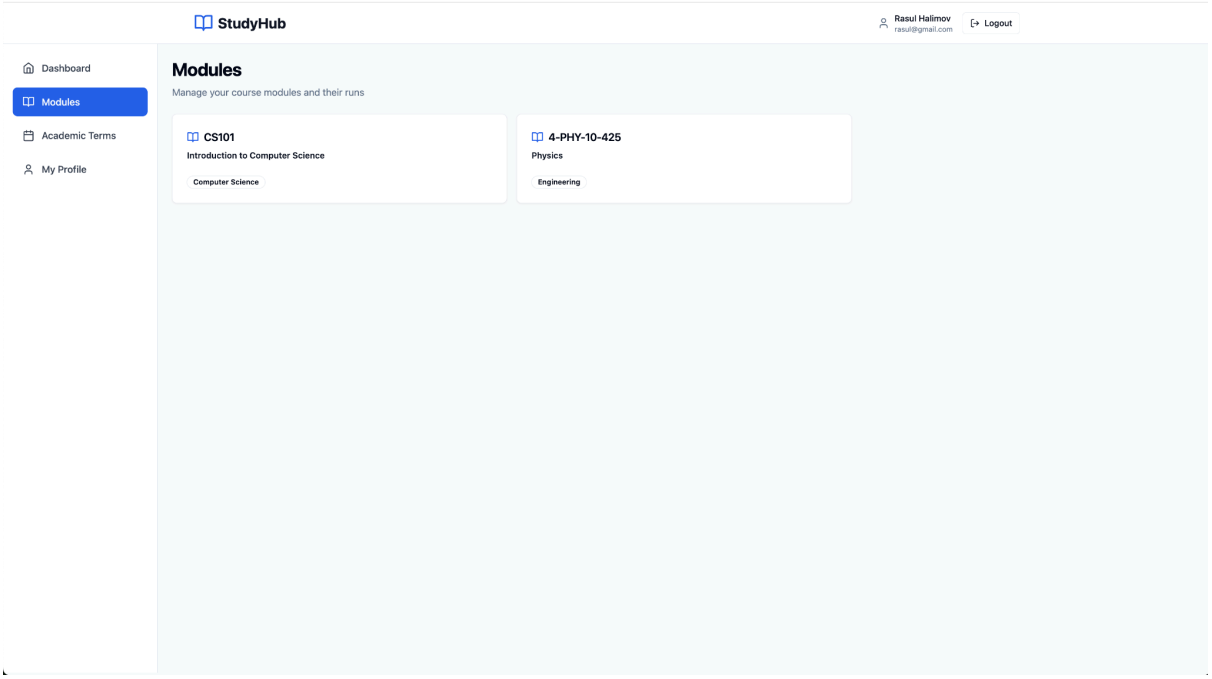
## Risks & Mitigation Strategies

| Identified Risk | Impact | Mitigation Strategy |
|---|---|---|
| **High AI Latency** | Long wait times for students when calling the Gemini API. | **Asynchronous Processing:** Using RabbitMQ to decouple the request from the generation process, ensuring the UI remains responsive. |
| **S3 Cost/Limit Overruns** | Frequent redundant uploads of the same textbook files. | **Data Integrity (Hashing):** The StorageObject entity uses a **Hash** to identify duplicate files, allowing multiple Resources to point to a single physical file in S3. |
| **Single Point of Failure** | If the Go Backend crashes, the entire app goes down. | **Reverse Proxy & Stateless Design:** Using Nginx allows for horizontal scaling (running multiple Go instances) while Redis manages shared session state. |

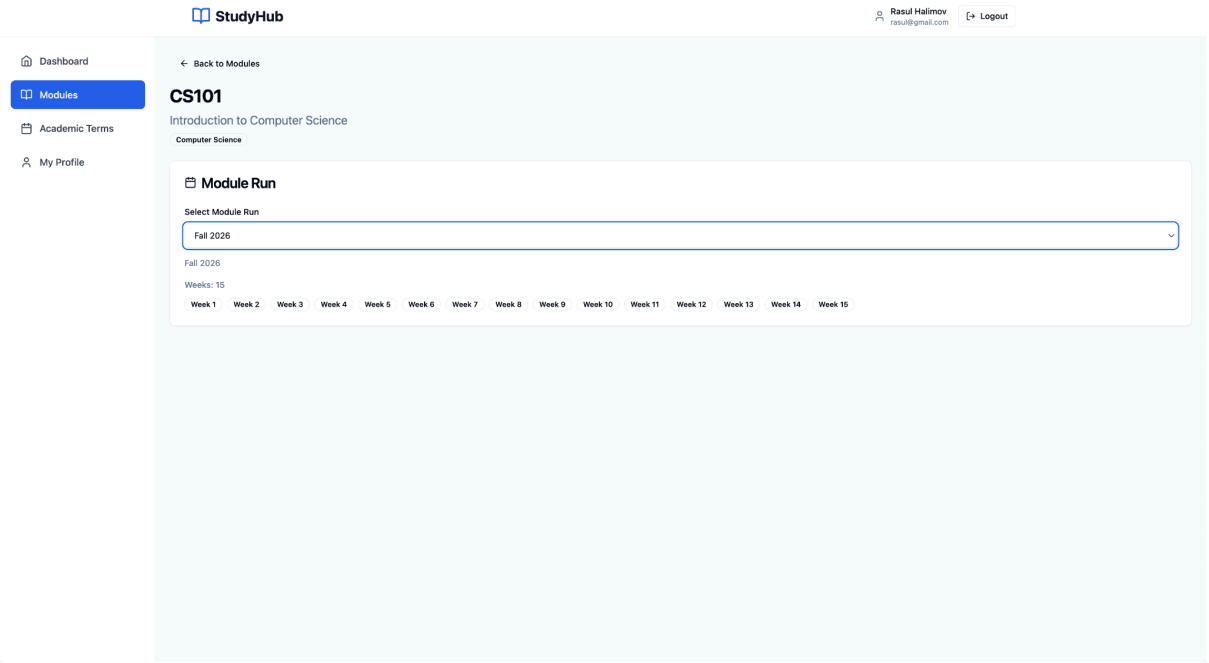
| | | |
|---|---|---|
| **Data Relationship Complexity** | Managing how resources link to different weeks and modules. | **Relational Integrity:** Using PostgreSQL with junction tables like `WeekResource` and `UserResources` to ensure clean Many-to-Many mappings. |

## UI/UX (last week we forgot to include):

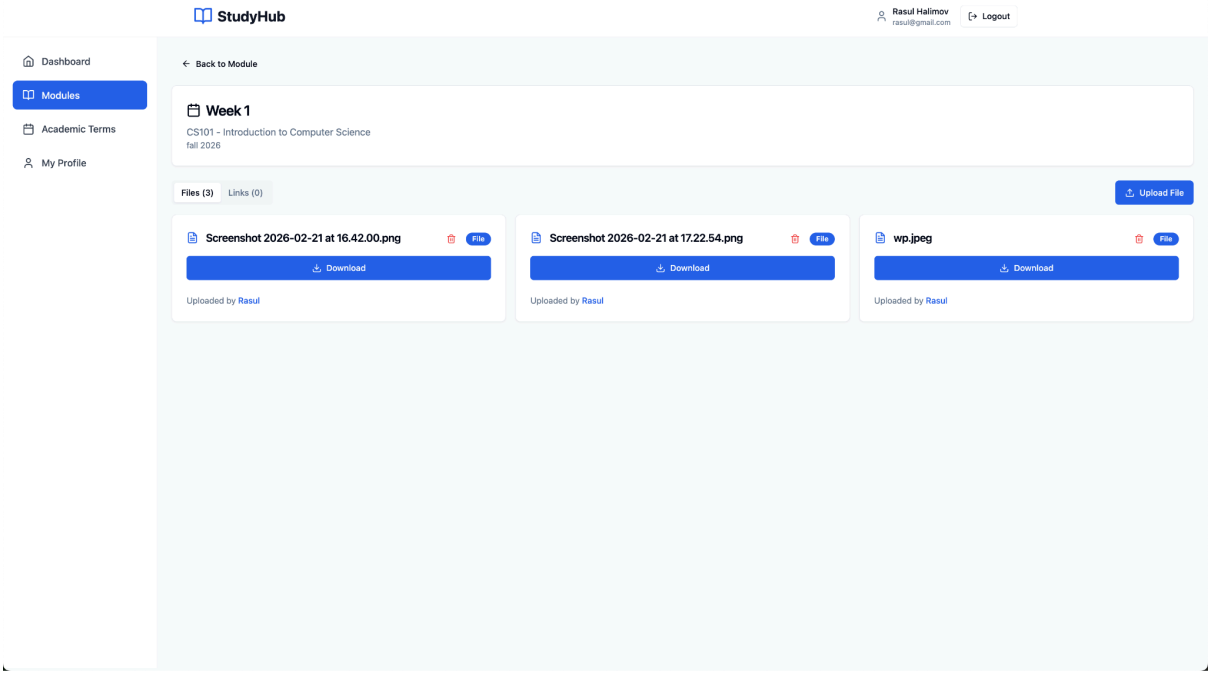
### 1. Main dashboard view for authenticated Users




### 2. View of the Module



## 3. View of the Week




Document Prepared By:

Nazarbek Ahmadov,
Abdullox Bahromjonov,
Usmon Karimov.